

AI Expense Tracker

Fourth Round Hackathon — Q&A Answer Key

React frontend • Python (FastAPI) backend • AI layer (auto-categorize + insights)

For interviewer / evaluator use only

Project Brief

Candidates build a full-stack AI-powered Expense Tracker in 2 hours 30 minutes, then record video answers to 5 technical questions. Below is the expected scope followed by model answers and what to look for in each response.

Required Tech Stack

- Frontend: React.js (functional components + hooks).
- Backend: Python — FastAPI, with Pydantic schemas and a SQLAlchemy data layer.
- Database: SQLite (fastest) or PostgreSQL.
- AI layer: a scikit-learn classifier OR an LLM API call — either is acceptable.

Required Features

- CRUD: Add, View, Edit, Delete expenses (amount, category, description, date).
- Filter by category and/or date range.
- Dashboard summary: total spent + per-category breakdown.
- AI auto-categorize: POST /api/ai/categorize predicts a category from the description.
- AI insights: GET /api/ai/insights returns a natural-language spending summary + one tip.

Expected REST Endpoints

GET	/api/expenses	list (optional ?category= filter)
POST	/api/expenses	create
PUT	/api/expenses/{id}	update
DELETE	/api/expenses/{id}	delete
GET	/api/expenses/summary	total + per-category totals
POST	/api/ai/categorize	{description} -> {category, confidence}
GET	/api/ai/insights	-> {summary, tip}

Video Q&A — Model Answers

System Architecture • Medium

1. Walk us through your AI Expense Tracker architecture. How does data flow from the React frontend to the FastAPI backend and back?

Model answer — key points:

- React component triggers an action (form submit / page load).
- A fetch/axios call hits a FastAPI route (e.g. POST /api/expenses).
- FastAPI validates the body with a Pydantic schema, then a service/CRUD function uses a SQLAlchemy Session to read/write SQLite.
- The ORM object is serialized back through a Pydantic response model to JSON.
- React updates state (useState) and re-renders the list / dashboard.

What to look for: Candidate clearly names each layer and where validation and the DB session live. Strong answers mention CORS config for the dev server and separation of routers vs. business logic.

Python & FastAPI • Medium

2. Explain your FastAPI endpoints. How did you structure routers, Pydantic schemas, and the data-access layer, and why?

Model answer — key points:

- APIRouter groups routes (expenses router, ai router) included from main.py.
- Pydantic models: ExpenseIn (request) and ExpenseOut (response, from_attributes=True).
- SQLAlchemy model Expense + a get_db() dependency that yields/closes a Session.
- CRUD done via db.query / db.get / db.add / db.commit; 404 raised with HTTPException.
- Separation keeps routes thin and testable; schemas give automatic validation + docs.

What to look for: Look for correct dependency injection (Depends(get_db)), proper status codes (201 create, 204 delete), and understanding of why Pydantic is used instead of raw dicts.

React Frontend • Medium

3. How did you manage state and API communication in React? Walk through your components and loading/error handling.

Model answer — key points:

- useState for expenses/form; useEffect to load on mount.
- A small api.js helper wraps fetch and centralizes the base URL and error checks.
- Component hierarchy: App -> ExpenseForm, ExpenseList, Insights/Dashboard.
- Loading and error flags drive conditional rendering (spinner / message).
- After create/update/delete, the list is re-fetched (or optimistically updated).

What to look for: Candidate handles the failure path (backend down / non-200), not just the happy path. Bonus: debouncing the AI category suggestion, controlled form inputs.

4. Explain the AI feature end to end. What model or API did you use, what is the input, and what is the output?

Model answer — key points:

- Auto-categorize: description text in -> predicted category (+ confidence) out.
- scikit-learn path: TfidfVectorizer + MultinomialNB trained on labelled descriptions; model.predict on the incoming text.
- LLM path: prompt the model with the description and the allowed category list, parse the returned category.
- Insights: aggregate expenses by category, then generate a natural-language summary + one saving tip (rule-based, or LLM-summarized).
- React consumes it: on description blur, call /api/ai/categorize and pre-fill the category dropdown.

What to look for: The AI must actually run and change behavior — not a hardcoded string. Candidate should explain training data / prompt, and a fallback (default to 'Other') when confidence is low or the API fails.

5. How would you add JWT authentication to this app? What changes in FastAPI and in React?

Model answer — key points:

- FastAPI: a /auth/login endpoint that verifies credentials and returns a signed JWT (python-jose).
- Hash passwords (passlib/bcrypt); store users in the DB.
- OAuth2PasswordBearer + a get_current_user dependency that decodes the token and is added to protected routes.
- Scope expenses to the authenticated user (user_id foreign key).
- React: store the token (localStorage), send Authorization: Bearer <token> on every request, redirect to login on 401.

What to look for: Understands the token lifecycle end to end and that authorization (per-user data) matters, not just authentication. Bonus: token expiry/refresh, not trusting the client.

